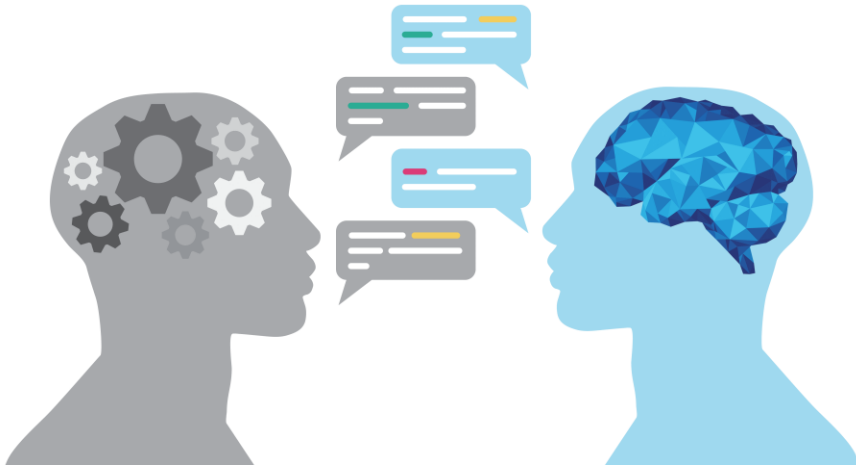




Road to ChatGPT: Basics of AI in Language Processing Day 2 Part 2

A/Prof Edmund Low
National University of Singapore
edmundlow@nus.edu.sg

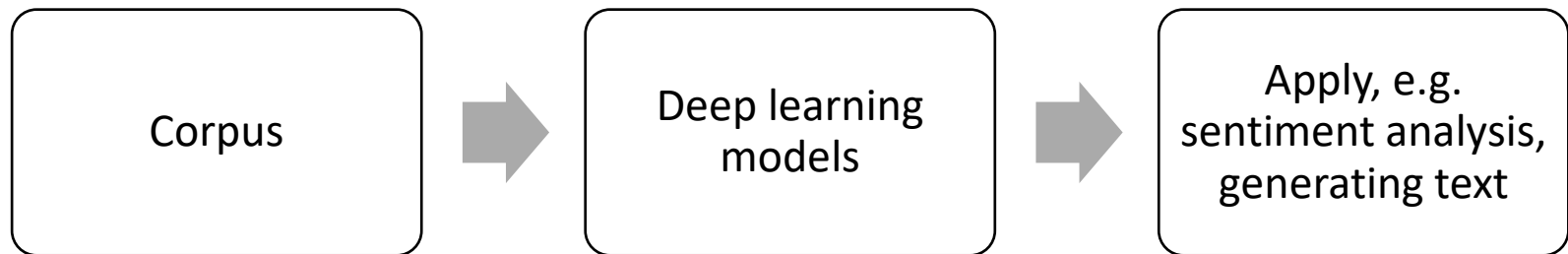
Agenda – making use of our processed data

- Predicting sentiment from words
- Embeddings – accounting for words that are closely related
- Improving model fit

Making use of our processed data

- Now that we have encoded sentences in our reviews to vectors
- Let's see if we can use it for an application in language processing, say in predicting the sentiment of a movie

What we will be doing



Building a model for
predicting sentiment

Training and evaluating our model

- How do we train our neural network model to be able to predict the sentiment of a movie review?
- How do we know that the model is giving us accurate results when applied to a new review?
- These aspects involve splitting the data we have into three sets

Training and evaluating our model



**e.g. the first
10,000 reviews in
our dataset**

**e.g. the next
10,000 reviews in
our dataset**

**e.g. the last
10,000 reviews in
our dataset**

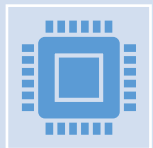
Training and evaluating our model



Training – we train the model on the training data



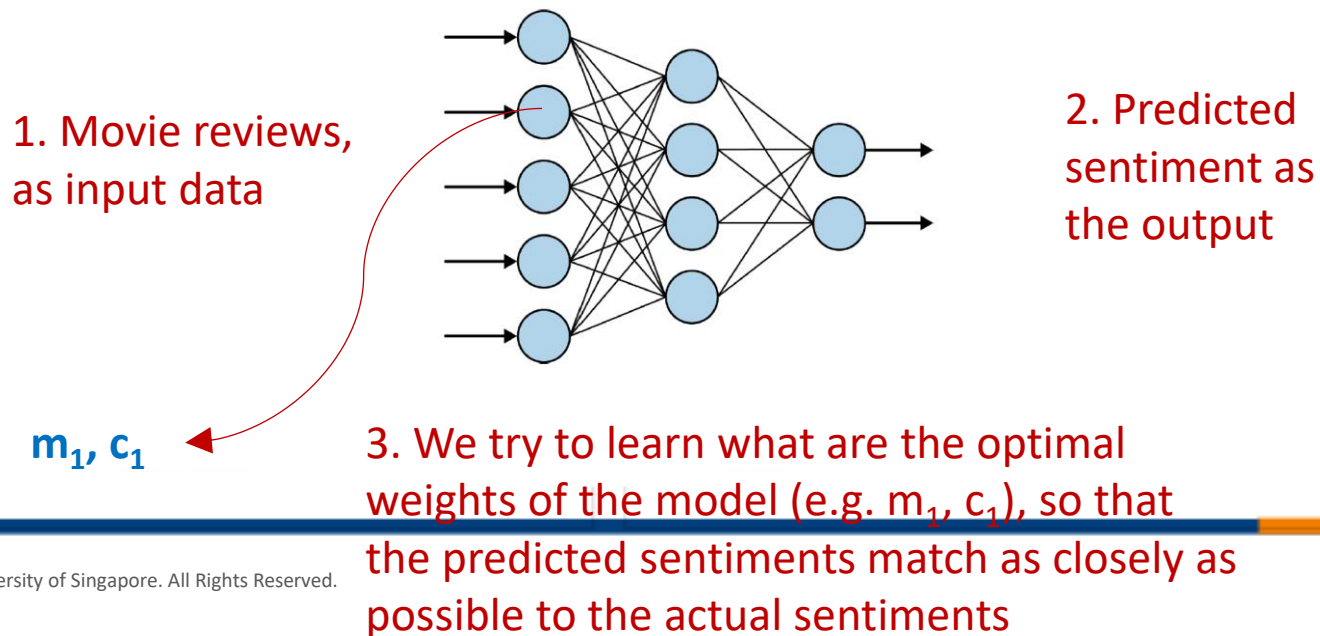
Validation – we evaluate the model on the validation data



Test – when our model is ready, prior to deploying it for actual applications, we test it one last time on the test data

Training data

- Given a model configuration (e.g. X number of layers, Z neurons in a layer etc), we “learn” the parameters (weights) of the model that will best link the inputs to the outputs



Overfitting

- There is a risk of overfitting to the training data
- Which means that our model is able to predict the sentiment of the movies in our training dataset perfectly, but fails when it comes to reviews it hasn't seen before
- We try to guard against overfitting by testing the model on an independent test set, containing data it hasn't seen before, prior to deployment

Why not just a train and test set?

- A part of model development is also on tuning its configuration, e.g. choosing the number of layers, the size of the layers etc
- These choices are known as the hyperparameters of the model, which are different from the model parameters – the weights of the neural network that we talked about earlier
- How do we know which configuration is “best”?

Validation data

- We use the model performance on the validation data as a way to tune these hyperparameters
 - Model trained with two layers : what's the performance on the validation set?
 - Model trained with three layers : what's the performance on the validation set?
- Even though we are not training directly using the validation data, we are adjusting the model configuration based on the validation data

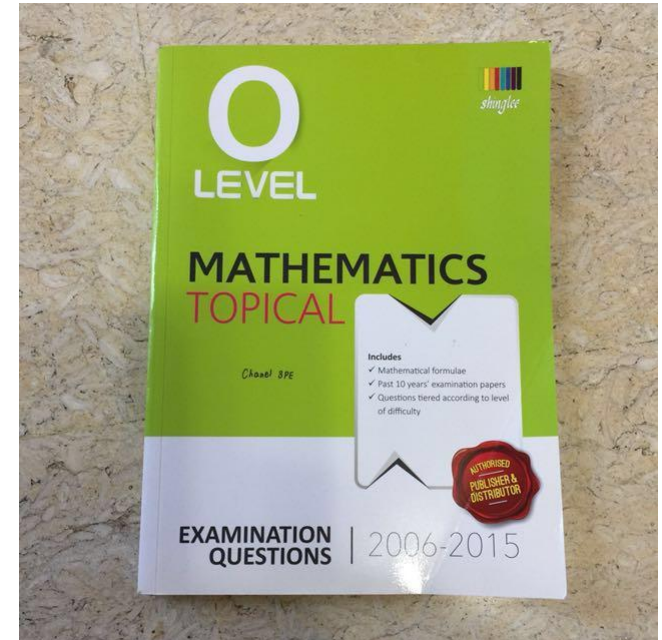
Validation data

- So we will keep tuning our hyperparameters using our validation data
- Therefore eventually, we will overfit to the validation data as well, through this repeated exposure
- Hence, we need the independent test set at the end of the day, to assess overall model performance

Training and evaluating...in school!



Training set – Homework



Validation set – Past year exams



Test set – Actual exam

A further note on assessment

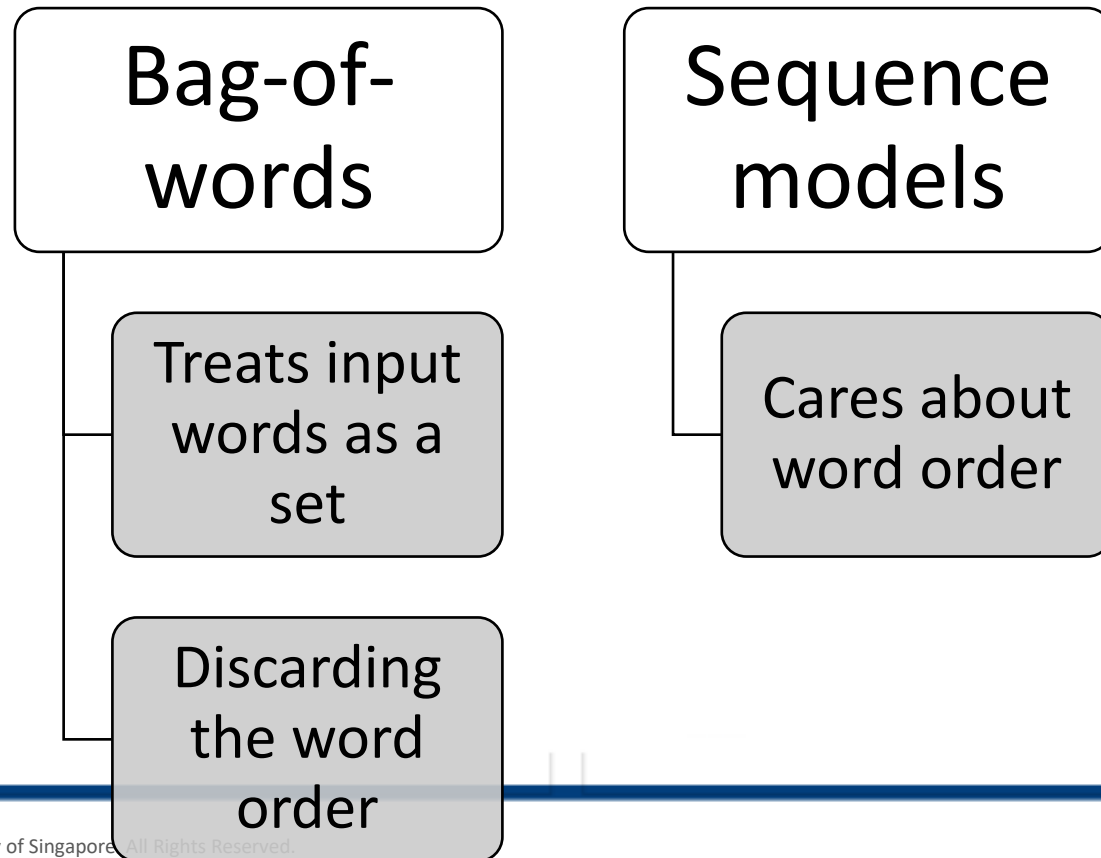
- Before we start training any model, we should think about any trivial baselines that might be applicable to our problem, which we should try to better
- For a model predicting sentiment of a movie review, what would make sense as the baseline performance?

Hands on

- Let's go to Kaggle now
- Create a new notebook and import the one called "Class2-3" – rename the notebook if you wish
- Let's go through the notebook

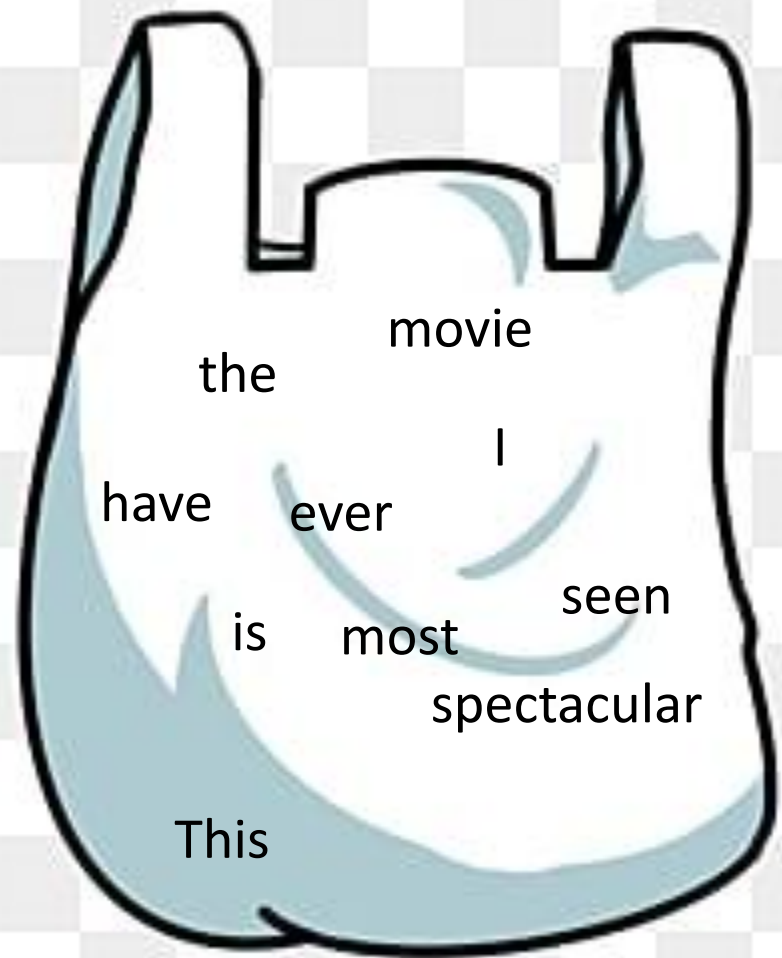
Bag-of-words models

- There are two kinds of text processing models



Bag-of-words models

- Consider our sentence: “This movie is the most spectacular I have ever seen”
- In this approach, we are just looking at the jumbled words, with no order, in a “bag”
- Let’s return to our notebooks to build this model



Building our first model

- In our first model, we have a single neuron learning a linear relationship

X

Learn (m, c)

$$Y = mX + c$$

X

$m = 3, c = -2$

$$Y = 3X - 2$$

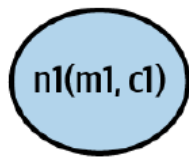
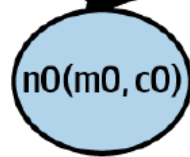
We have a more complicated problem now...

- Each of our movie reviews contains 600 integers, each of which can take on one of 20,000 possible values (our vocab size); this forms our X in this case
- For Y , each of our movie reviews is either positive or negative, so it takes on 1 or 0
- One neuron is no longer enough; but we can have multiple neurons working together

Our more complicated problem, represented as a neural network



We have inputs (our vectorized movie reviews) going into each neuron in our layer

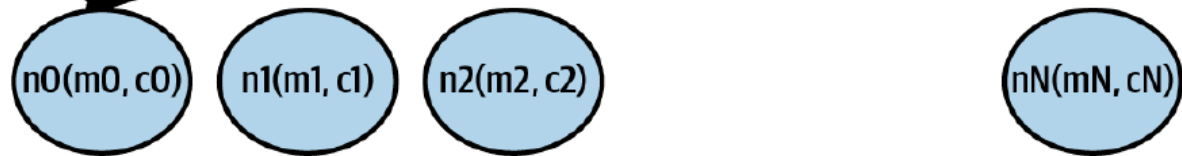


$$f(m0, c0) + f(m1, c1) + f(m2, c2) + \dots + f(mN, cN) = 1$$

Negative (0)

Positive (1)

Our more complicated problem, represented as a neural network



$$f(m_0, c_0) + f(m_1, c_1) + f(m_2, c_2) + \dots + f(m_N, c_N) = 1$$

Negative (0)

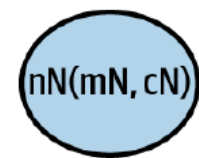
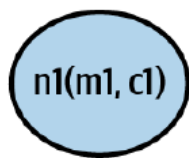
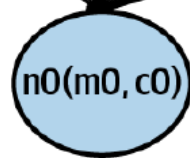
Positive (1)

Each neuron is going to transform the input (a vectorized movie review), based on its parameters, m and c

Our more complicated problem, represented as a neural network



We then have a combined function of all the neuron outputs, to see if we can match the predicted to the actual answer, positive / negative sentiment



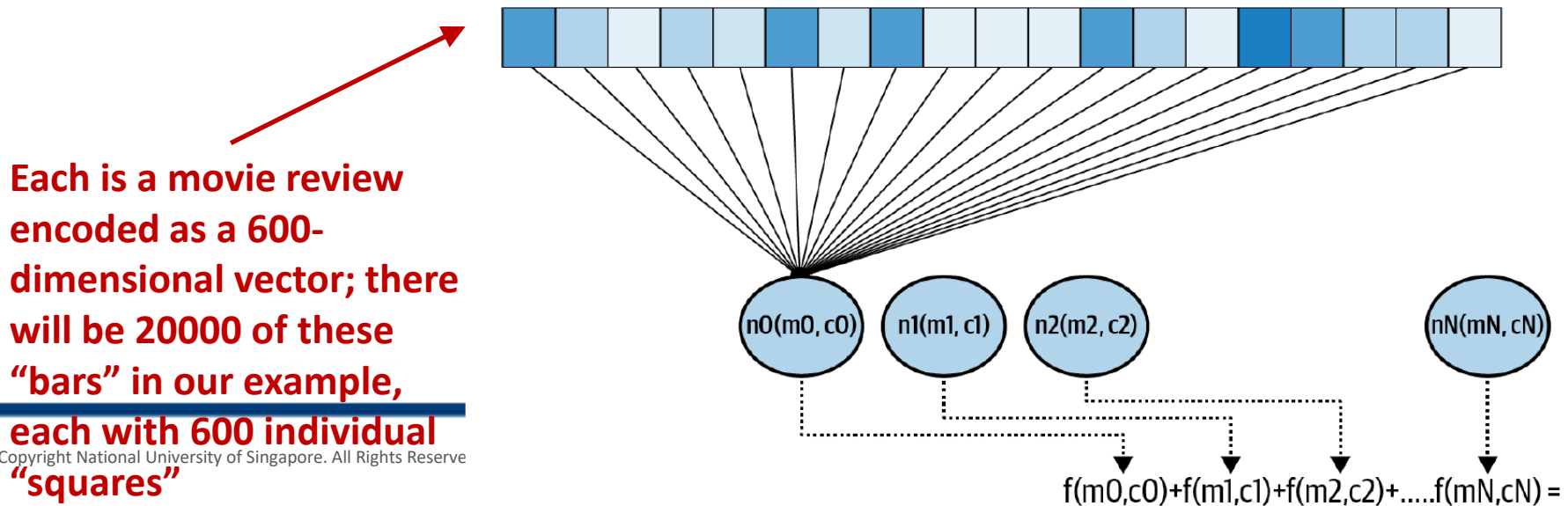
$$f(m0, c0) + f(m1, c1) + f(m2, c2) + \dots + f(mN, cN) = 1$$

Negative (0)

Positive (1)

Building our model

- `inputs = keras.Input(shape = (600,))`
- This defines the inputs that will be loaded into a layer of neurons when we train a neural network

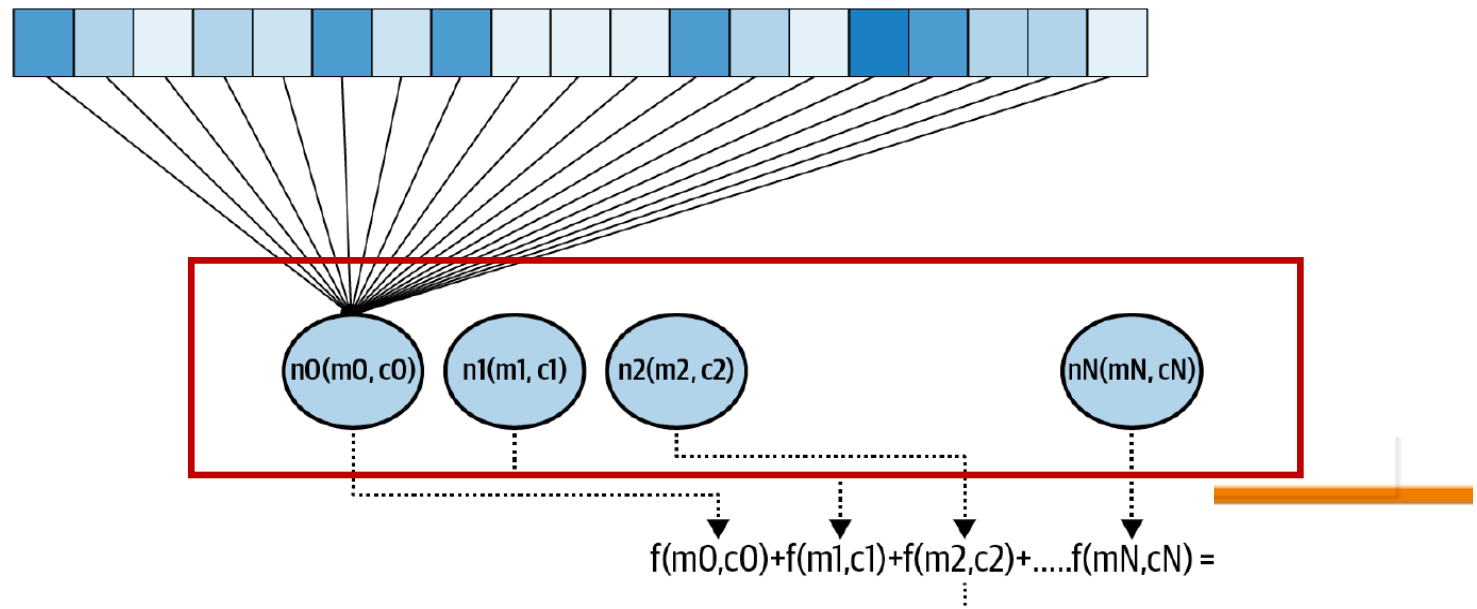


Building our model

- In this case 600-dimensional vectors representing each of our tokenized reviews
- In the diagram, we see the inputs loaded into just the first neuron, but they are in fact loaded into each neuron in the layer

Building our model

- `x = layers.Dense(16, activation = 'relu')(inputs)`
- The line above specifies we want a Dense layer of 16 neurons; this is our middle layer as shown below



Building our model

- Middle layers are sometimes called “hidden layers”, since they are situated between the input and output layers, and aren’t seen by the user
- In the code, we are asking for 16 neurons to have their parameters randomly initialised
- Why 16? Why not another number?

Building our model

- Choosing an appropriate number of neurons per layer is one of the things we might do as part of hyperparameter tuning
- Hyperparameter: a value that is used to control the training
- Parameter: Internal values of the neurons that get trained/learned

Building our model

- For each neuron, across all incoming movie reviews, there will be a total of 601 weights (600 values of “m” and 1 value of “c”) – these weights are the same for all reviews coming into a neuron
- So looking at a particular review, say review #1, coming into neuron #1, we get:
- $y_1 = m_1x_1 + m_2x_2 + \dots m_{600}x_{600} + c_1$
- Where $x_1, x_2, \dots x_{600}$ are the integers representing our tokenized movie review #1
- y_1 is the intermediate output for review #1 from neuron #1, and $m_1, m_2, \dots, m_{600}, c_1$ are the set of parameters from neuron #1

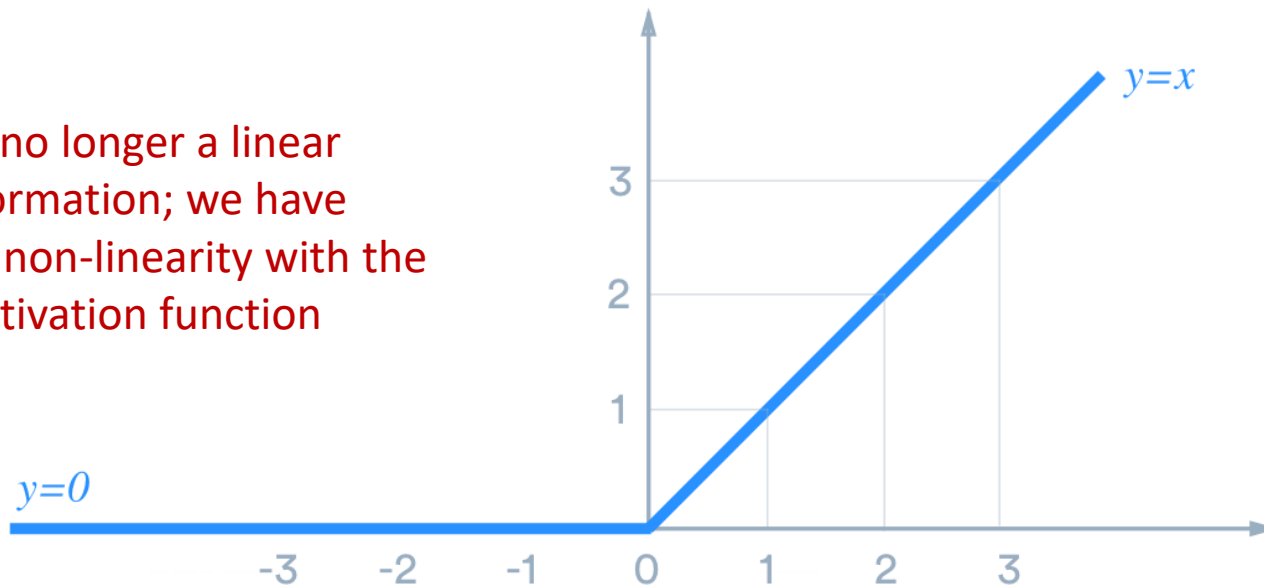
Building our model

- Across 16 neurons in the layer, the total number of trainable parameters in the layer becomes $16 \times 601 = 9616$
- We notice an activation function being specified in this layer, 'relu'
- The activation function adds non-linearity to the transformation

Building our model

- In this case, for relu, which stands for “rectified linear unit), it “zeroes out” negative values in the intermediate outputs from this dense layer

This is no longer a linear transformation; we have added non-linearity with the relu activation function



Building our model

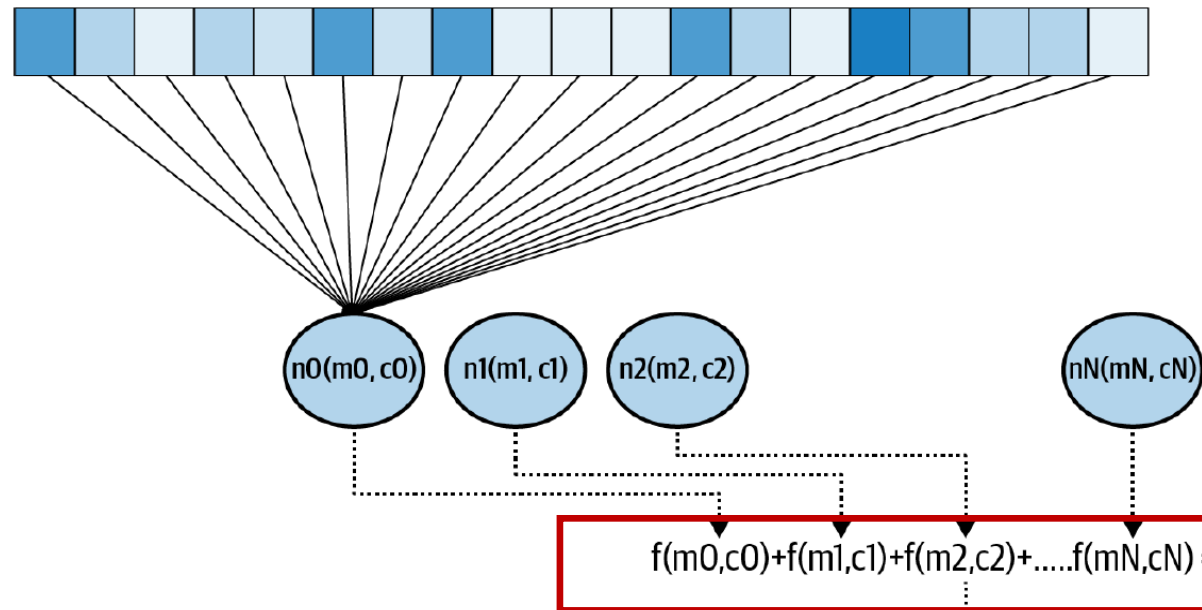
- relu is a common choice for the activation function for middle layers
- In our case, it helps because we don't want negative values being passed to the next layer, where we will be summing up outputs to get a probability (which cannot be negative)

Building our model

- `outputs = layers.Dense(1, activation = 'sigmoid')(x)`
- Finally, we have another Dense layer, which is the output layer; this has just one neuron, because we are only trying to predict between two classes (positive and negative)
- The neuron will give a probability that a review is positive, and depending on the value, we can classify as positive or negative

Building our model

- The output layer is going to take as inputs, all the intermediate results from the previous layer, $y_1, y_2, \dots, y_{16}$ (one per neuron)
- It will then apply its own set of weights to these inputs

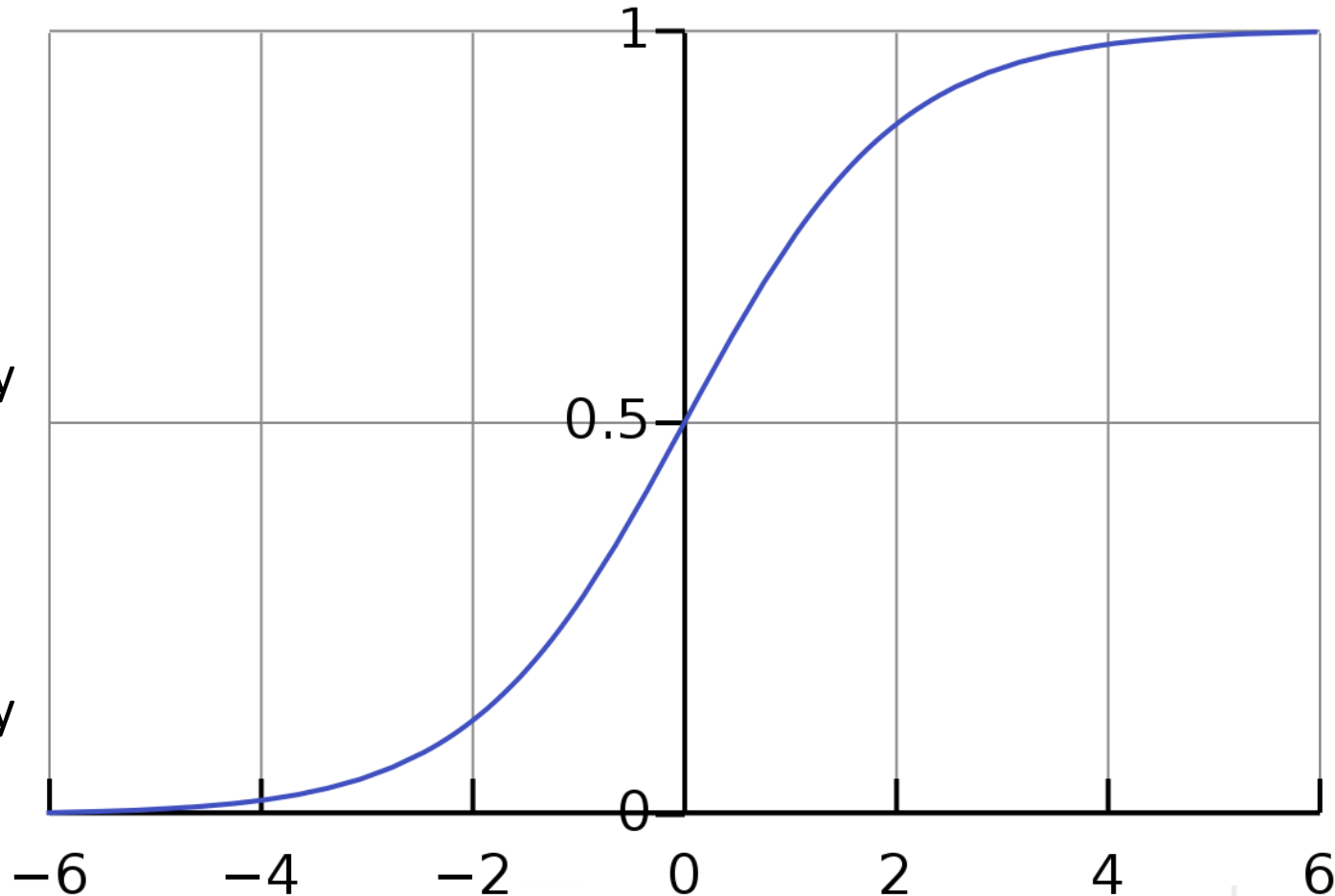


Building our model

- Say for review #1
- $\text{output}_1 = m_1 y_1 + m_2 y_2 + \dots m_{16} y_{16} + c_1$
- Hence, the total number of trainable parameters is 17 for this output layer
- The output goes through another activation function, “sigmoid”, which compresses values within the $[0, 1]$ range, and hence outputs a value that would make sense as probability

Building our model

- $y = \frac{1}{1+e^{-x}}$
- What happens when x is very large and positive?
- What happens when x is very large and negative?



Model compilation step

- `model.compile(optimizer = 'rmsprop',`
- `loss = 'binary_crossentropy',`
- `metrics = ['accuracy'])`
- Using the `compile()` method, we configure the model for training; we specify the optimizer, loss function and metric
- Recall that the loss function is the one we try to minimise during training

Model compilation step

- Commonly for classification problems with two classes, like in this case for positive and negative reviews, “binary crossentropy” is used
- The metric tracks incorrect labeling of the data class by a model, and penalizes the model for such deviations in classifying the label
- For the optimizer, we use a variant of the gradient descent that we discussed earlier, “rmsprop”, that has been shown to perform well for neural networks

Model compilation step

- For the metric, we chose “accuracy”, which looks at the percentage of correct predicted labels
- The metric is used as a gauge of the overall performance of the model
- Why not use “accuracy” as the loss function?

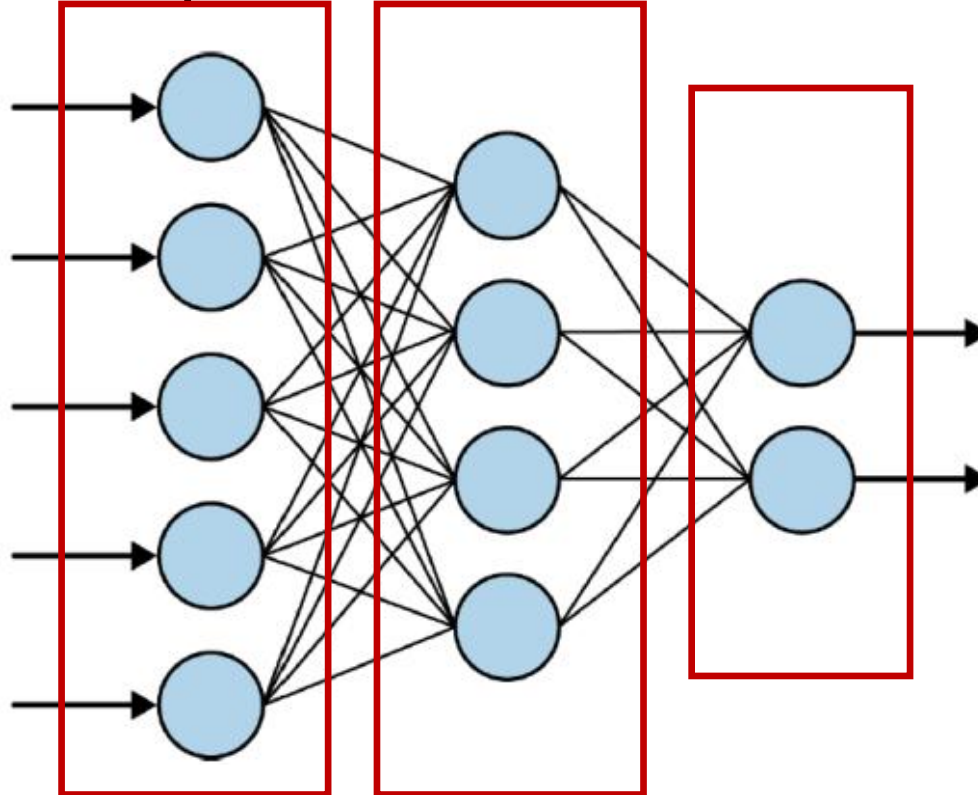
Model compilation step

- Recall from our first class that the loss function has to be differentiable, so that its gradient can be found for gradient-based optimization
- The metric of accuracy isn't differentiable – either a label is accurate, or it isn't (not continuous and therefore differentiable)
- Now that we have configured our model for training, let's go back to our notebooks

Training the model

- `model.fit(arr_train_reviews, arr_train_labels, epochs = num_epochs, batch_size = 32, validation_data = (arr_val_reviews, arr_val_labels), callbacks = callbacks)`
- When we call the `fit()` method, the model will start to iterate on the training data (`arr_train_reviews`, `arr_train_labels`), in mini-batches of 32 samples (i.e. 32 reviews each time), 100 times over (the number of epochs)

Forward pass



Output layer :
Takes the
intermediate
outputs and
ultimately
generate
probabilities
of the 32
movie
reviews being
positive
(Class 1)

Input layer : Batches of 32
movie reviews gets fed into our
neural network each time

Hidden layer : Initial randomised
weights for the model transform
the input data, before passing
their intermediate outputs to the
output layer

Using these
probabilities,
the loss is
calculated

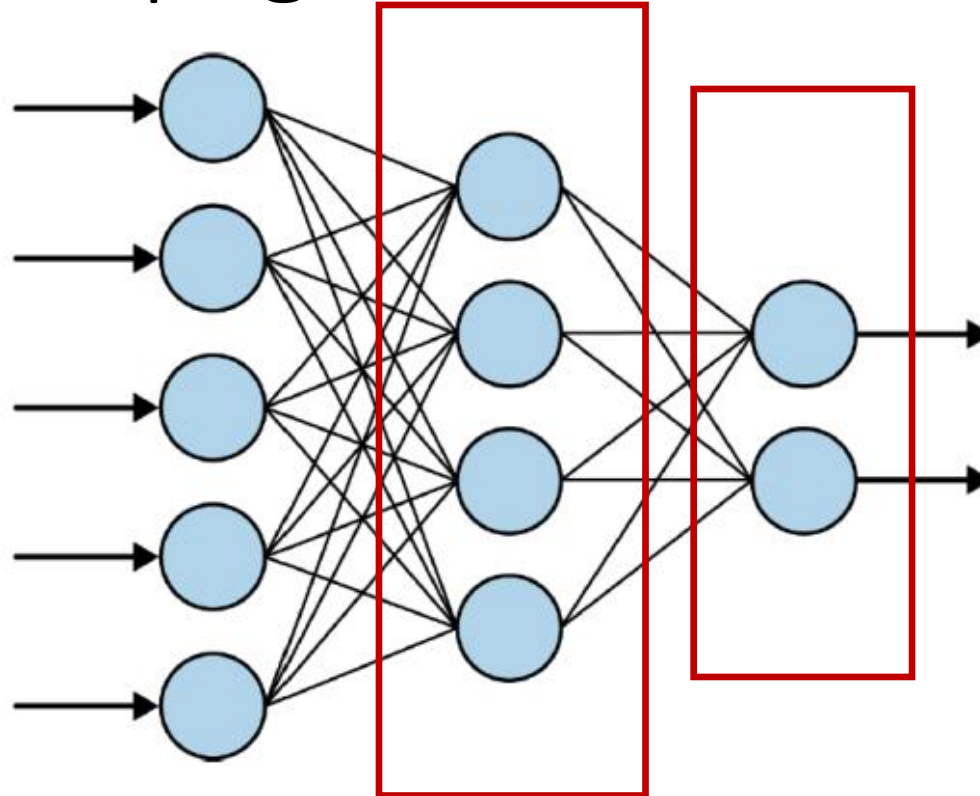
Training the model

- For each batch, the model will compute the gradient of the loss, with regards to the current set of weights, using the backpropagation algorithm
- The optimizer will then implement the specifics of gradient descent, adjusting the weights, based on the gradient and learning rate (see Class 1) → **this is one adjustment**
- After we are done with batch #1, we move on to the next batch of 32 movie reviews to be fed into the model for epoch #1

Training the model

- So for each epoch, the weights will be adjusted 625 times (20000 movie reviews total in the training set, split into 32 batches)
- Over 100 epochs, our weights will be adjusted a total of 62 500 times, hopefully yielding a low loss, accurate model here

Backpropagation



Output layer :
Loss will
propagate
back to the
hidden layer

Hidden layer : backpropagation
algorithm calculates the gradient of
the loss with respect to each weight;
optimizer adjusts the weight based on
the gradient and learning rate

Training our model

- `model.fit(arr_train_reviews, arr_train_labels, epochs = num_epochs, batch_size = 32, validation_data = (arr_val_reviews, arr_val_labels), callbacks = callbacks)`
- After each epoch, with the set of weights at this point, we also apply our model on the validation set to compute the loss and accuracy
 - This validation loss and accuracy can be compared across various runs with different configurations for hyperparameter tuning
- The callback object allows us to checkpoint and save the model at various points during training

Callbacks

- We define our callback object as follows:
- `callbacks = [`
- `keras.callbacks.ModelCheckpoint('bag_of_words_model', save_best_only = True)`
- `]`
- Here, our ModelCheckpoint callback is used in `model.fit()` (see prev slide) to save the model that has achieved the "best performance" so far

Let's go back to our notebooks, run the code and see the results

A silver stethoscope is positioned diagonally across the frame, with its chest piece at the bottom and its earbuds at the top right. The background is a plain, light gray.

How do we feel about the training accuracy of $\sim 50\%$?

Why do you think the performance is what we've observed?

Trying something else...

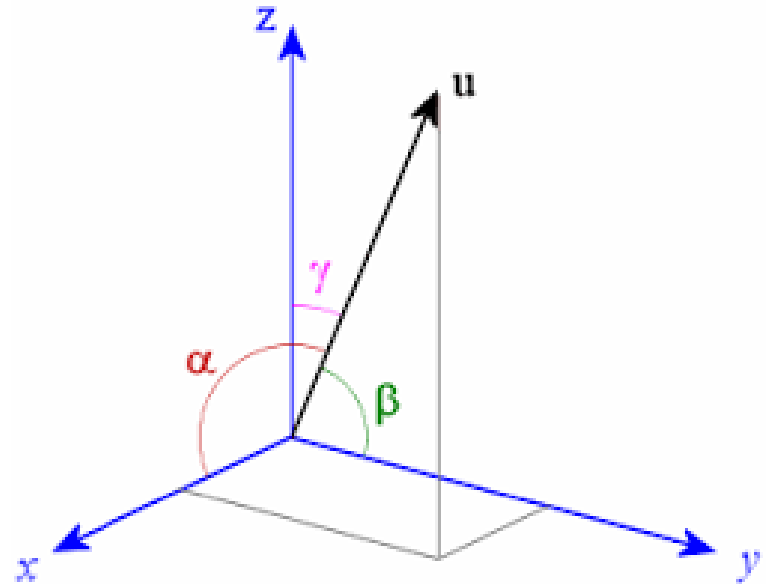
- The approach that we have taken before, was just to represent each word as a number
- This form of encoding doesn't really allow us to capture the meaning of a word
- For instance, representing "fantastic" as 41

Trying something else...

- While there's probably no absolute numeric encoding that will allow us to capture meaning, there are relative ones, and we will look at an example here...

Embeddings

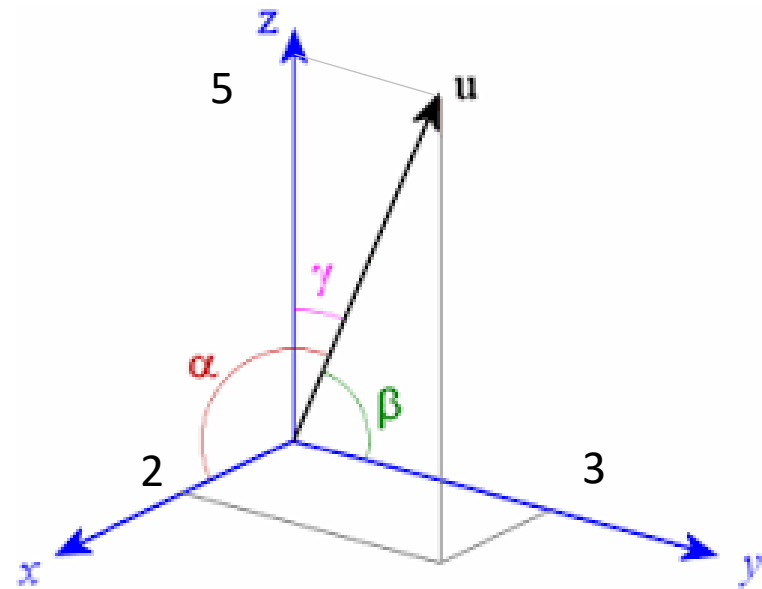
- The concept of embeddings refer to the use of vectors in high dimensional space to represent words, instead of single integers
- Recall that a vector has both magnitude and direction



Embeddings

- For example, the vector shown is one in 3-dimensional space, and can be written in the following notation:

$$\begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix}$$

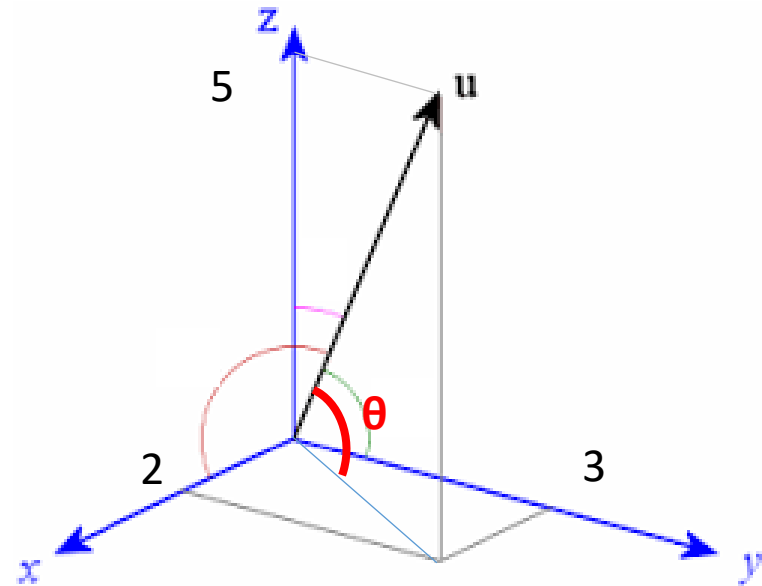


Embeddings

- Magnitude = $\sqrt{2^2 + 3^2 + 5^2} \approx 6.16$
- Direction (wrt to x-y plane):

$$\theta = \sin^{-1} \frac{5}{6.16} = 54.2^\circ$$

- We're going to use a vector instead of an integer, to represent each word

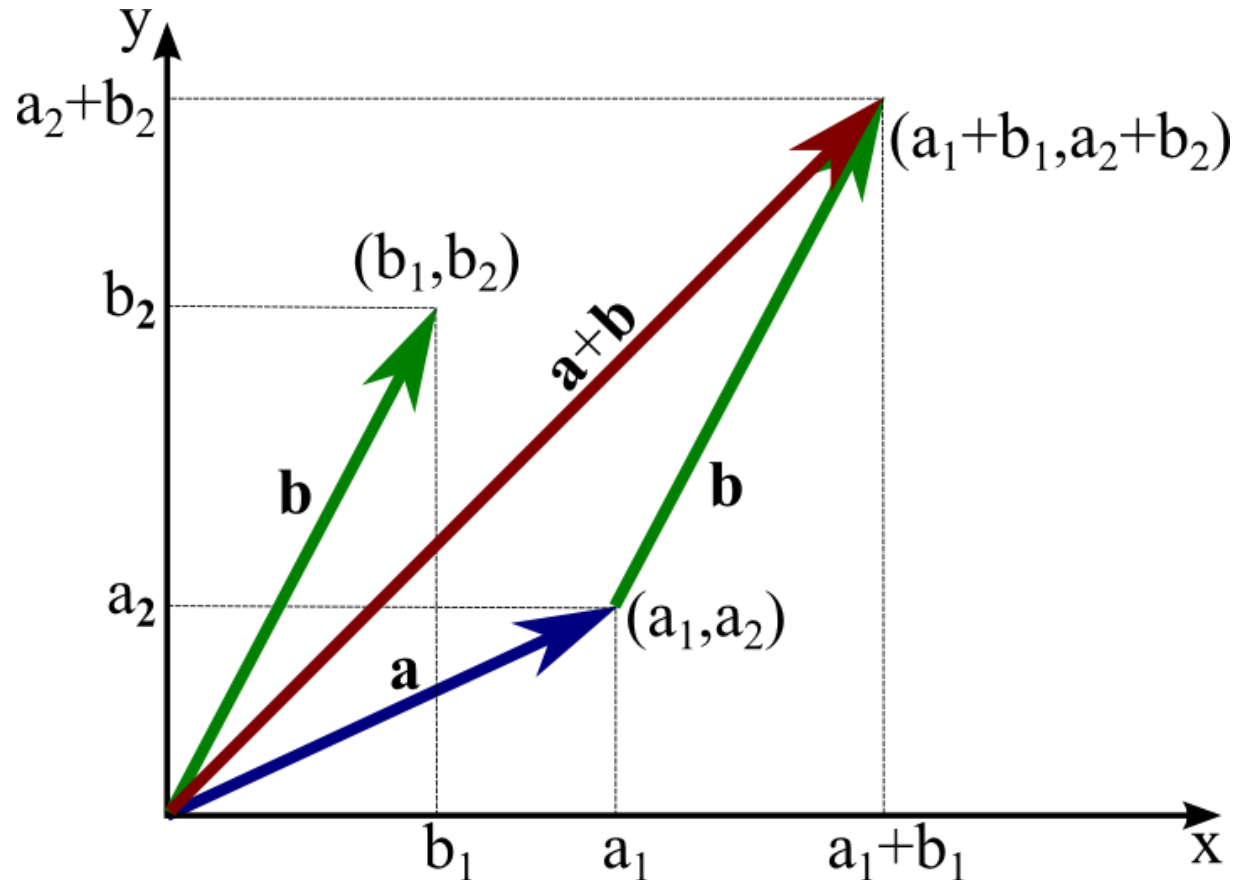


Embeddings

- The directions of the vectors representing each word will be learned by the model, based on the use of the words in the corpus
 - Related words will have similar directions
- Hence, when given a review, we can look at the directions of the word vectors making up the review, sum them up, and from the overall direction of the summation, infer the sentiment of the review

Example: Adding up 2D vectors

- Say vector **a** is (3, 2) and vector **b** is (2, 3)
- Adding up the two vectors, we get (5, 5)
- The summed vector is pointing in roughly the same direction as the two constituents



Embeddings

- So if “northeast” = positive sentiment, then summing up our positive word **a** with our positive word **b**, yields a two-word combo that is still overall positive in sentiment
- Because it’s still pointing in the northeast direction
- That’s the principle behind representing words as embeddings → to capture this “relatedness” behind words
- Words that are related will be pointing in similar directions for their embeddings



Embeddings

- Let's roll back a few steps to try to first visualise how meaning of words can be represented by numbers
- Take for example this simple positive review:
 - I absolutely love the performance in the movie
- And this simple negative review:
 - This was the worst movie I have seen this year

Embeddings

- What if we were to encode all the words in the positive review, with positive numbers, and all the words in our negative review, with negative numbers?
- {I : 1, absolutely : 1, love : 1, the : 1, performance : 1; in : 1, the : 1, movie : 1}
- {This : -1, was : -1, the : -1, worst : -1, movie : -1, I : -1, have : -1, seen : -1, this : -1, year : -1}

Embeddings

- Now, let's take a look at this new review:
- I have not seen a worst performance this year
- We could look at the scores for each word and add them up
- {I : 0 ; have : -1; not : [UNK], seen : -1, a : [UNK], worst : -1; performance : 1, this : -1, year : -1}
- What total will we get?

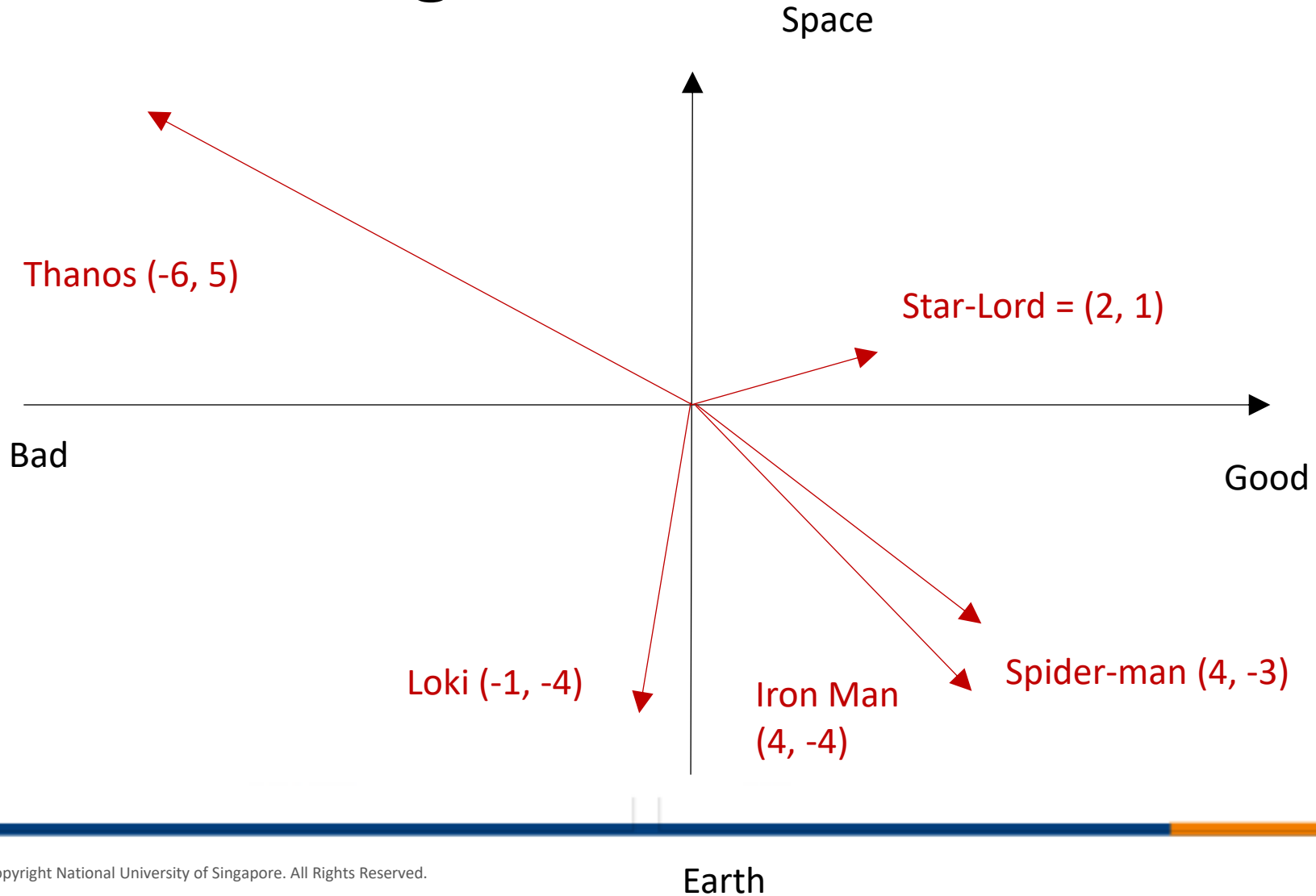
Embeddings

- Just a simple example to illustrate how some form of relative meaning of a word can be established, through its association with other words in the same “direction”
- i.e. moving all words in the positive review in the positive direction, and vice versa
- Then seeing for the new review, containing words from both positive and negative reviews, what is the overall direction after summing

Embeddings

- Let's try to first visualise now how meaning of words can be represented by higher dimensional vectors
- For example, assuming that we're looking at Marvel movie characters along the dimensions of whether they are good/bad, and whether they live in space/Earth
- With the length of the vector indicating how powerful the character is

Embeddings



Embeddings

- A common way to quantify the relatedness of two word embeddings (besides summing up the vectors) is to compute the dot product

- Between Spider-man and Iron Man:

$$4 \times 4 + (-3) \times (-4) = 28$$

- Between Spider-man and Thanos:

$$4 \times (-6) + (-3) \times (5) = -39$$

Embeddings

- As this example shows, by looking at multiple dimensions, we can discern real meaning in the words (Marvel character names here)
- Not really concrete definitions, but a relative meaning based the relation between the vector for one word, with another
- That's essentially what embedding is, a vector representation of a word that is learned via training in a neural network, rather than a number that is assigned

Embeddings in TensorFlow

- Embeddings is implemented using a layer
- This create a lookup table mapping an integer (the word) to an embedding table, which contains the coefficients of the vector representation of that word
- So in our Marvel example, the x and y coordinates would give us the embedding for a particular character

Embeddings in TensorFlow

- Of course, in an actual problem, we will be using more than 2 dimensions, as we will see in our hands-on
- We can thus view the direction of a vector as encoding the meaning of a word; words with vectors that are pointing roughly in the same direction can therefore be considered as related words
- The embedding layer will be initialised randomly at the start, i.e. the coefficients for the vectors will be randomised, and learned during training

Embeddings in TensorFlow

- When the training is done, we should have embeddings that roughly encode similarities between words, by way of the directions of the vector encodings
- Let's take a look at how this can be done in code
- Create a new notebook, import the notebook "Class2-4", rename it → then let's go through the notebook

Building our model

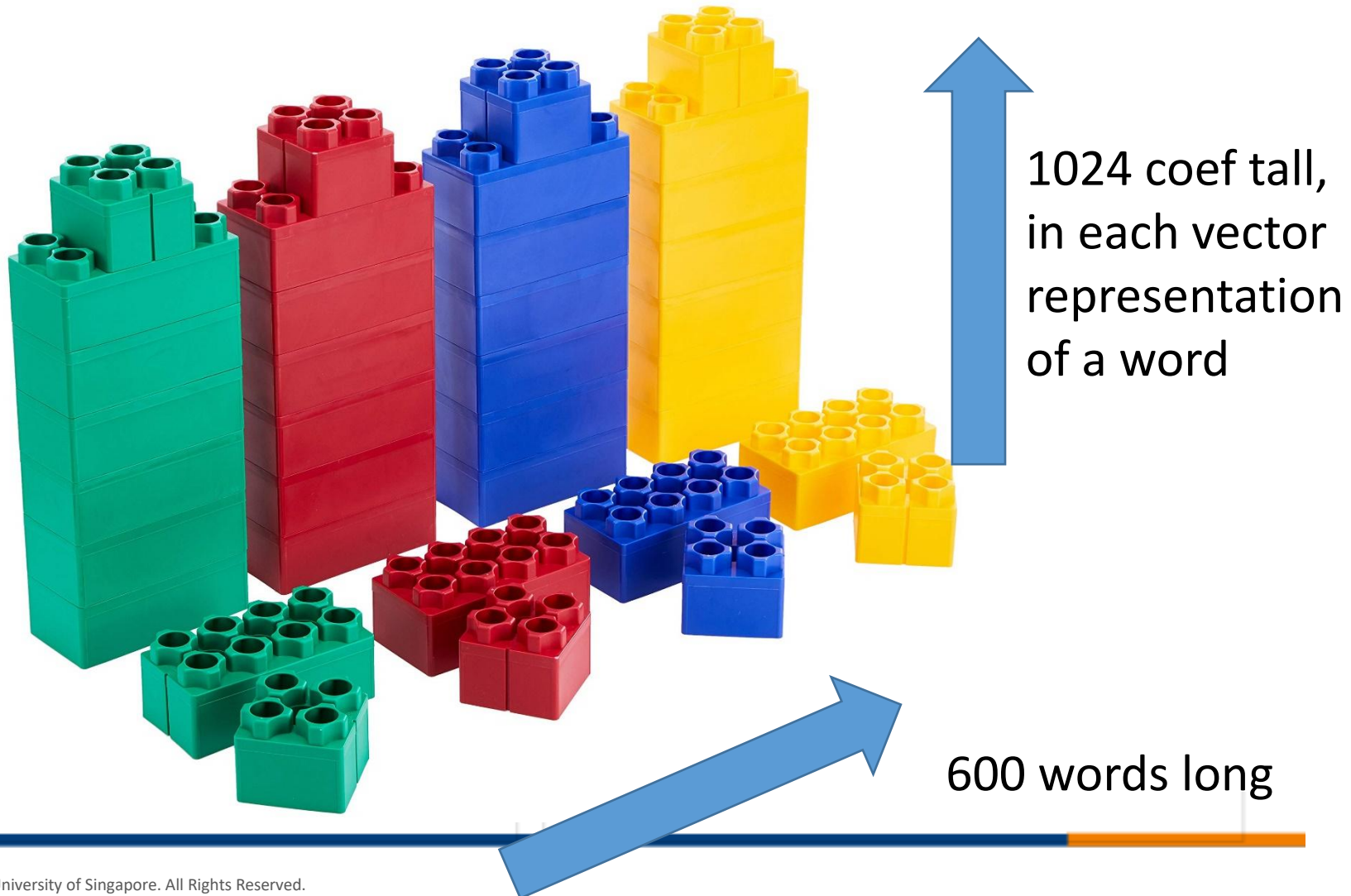
- `x = layers.Embedding(input_dim = vocab_size, output_dim = embed_dim)(inputs)`
- We create our embeddings layer, specifying the number of words in our vocab to be encoded as vectors, and how many dimensions should each vector be
- In our case, we have 20000 words, each encoded as a 1024-dimensional vector, which means we have 20,480,000 parameters to learn, for this layer

Building our model

- This is what we observe from the model summary, and the outputs coming out from this layer is going to have shape (None, 600, 1024)

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 600)]	0
embedding (Embedding)	(None, 600, 1024)	20480000
global_average_pooling1d (GlobalAveragePooling1D)	(None, 1024)	0
dense (Dense)	(None, 16)	16400
dense_1 (Dense)	(None, 1)	17
Total params: 20,496,417		
Trainable params: 20,496,417		
Non-trainable params: 0		

Building our model



Building our model

- `x = layers.GlobalAveragePooling1D()(x)`
- Here, we average all the vector representations of the words in each review, i.e. each review is going to end up being represented by just one aggregated 1024-dimensional vector
- Recall earlier that summing up the vectors can give use a sense of the overall direction, in this case, the sentiment of the review

Building our model

- Which is why the output shape has become (None, 1024) indicating each review is represented by a 1024-dimensional vector
- Since we are just pooling the vector representations, there are no parameters to learn here

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 600)]	0
embedding (Embedding)	(None, 600, 1024)	20480000
global_average_pooling1d (GlobalAveragePooling1D)	(None, 1024)	0
dense (Dense)	(None, 16)	16400
dense_1 (Dense)	(None, 1)	17
Total params: 20,496,417		
Trainable params: 20,496,417		
Non-trainable params: 0		

Building our model

- `x = layers.Dense(layer_dim, activation = 'relu')(x)`
- With this new representation, we feed, as before, into a 16-neuron hidden dense layer, before the final output neuron predicting whether it is class 1 or 0 (positive or negative)
- For the hidden layer, since the inputs are 1024-dimensional vectors, for each neuron:
- $y_i = m_1x_1 + m_2x_2 + \dots + m_{1024}x_{1024} + c_i$

Building our model

- There are 1025 parameters to train for each neuron, across 16 neurons giving us a total of 16400 for this layer
- The output shape from this hidden layer would be 16-dimensional ($y_1, y_2, \dots, y_{16}$), which goes into the output layer
- `outputs = layers.Dense(1, activation = 'sigmoid')(x)`

Building our model

- For the output neuron, it will have to fit 17 parameters, yielding a final probability as the result
- $z = m_1y_1 + m_2y_2 + \dots + m_{16}y_{16} + c$
- The value of z (after going through the sigmoid activation) will be the probability of the review being positive)
- Let's go back to the notebook and discuss the next code block for fitting the model

Let's review the results

- What does everyone think?

Let's also review the charts

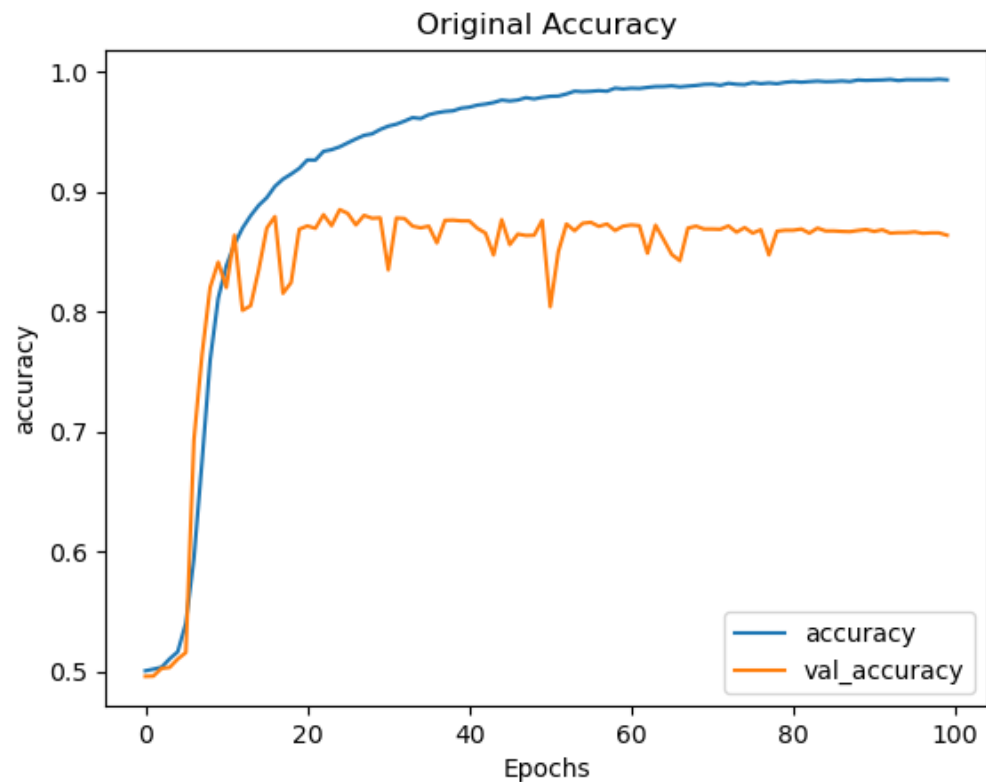
- But before we generate the charts, let's try to predict the results we'll get
- We are going to be look at how accuracy and loss is going to trend as the training proceeds
- How are the trends going to look like?

Let's also review the charts

- In addition, we are also going to see specifically how accuracy and loss for the validation set, trends with time
- How will we predict the trends to look like, compared to those for the training set?

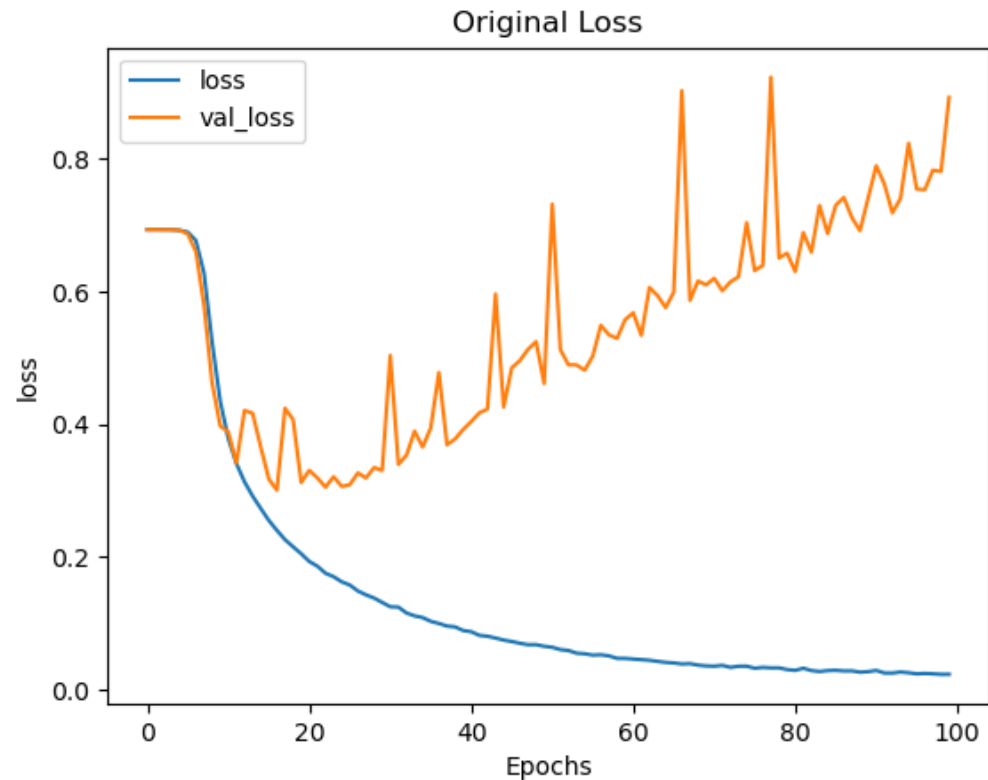
Let's also review the charts

What do we notice?



Let's also
review
the
charts

What do we notice?





Improving the model

Why do you think the loss chart as observed is a sign of overfitting?

How then might we improve our model? Let's explore several ways

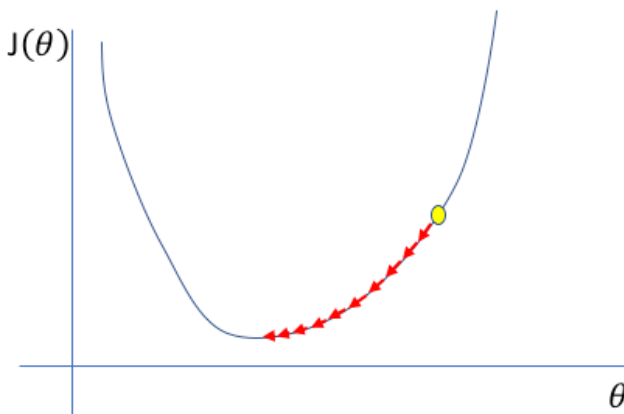
Learning rate

- One way is to adjust the learning rate
- Recall that the learning rate defines how big of an adjustment to make on the weights, given the gradient of the loss
- If we have too small of a learning rate, the model “learns” too slowly, and we might need a lot of epochs (and computational resources) before we get to the minimum loss

Learning rate

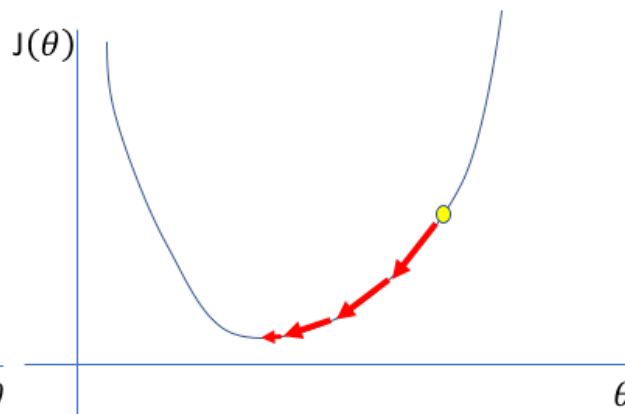
- However, if the learning rate is too big, we might learn too fast, and miss the minimum loss altogether; let's try in our notebooks

Too low



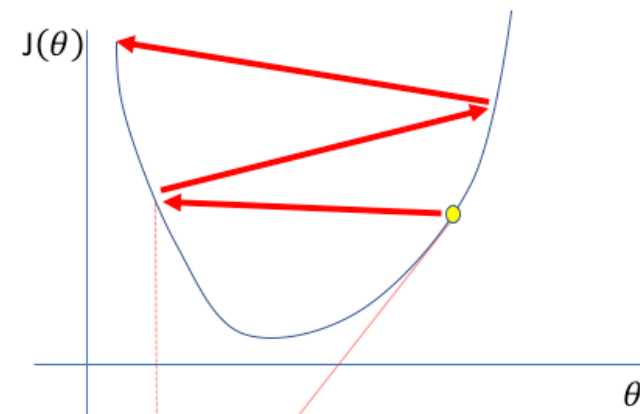
A small learning rate requires many updates before reaching the minimum point

Just right



The optimal learning rate swiftly reaches the minimum point

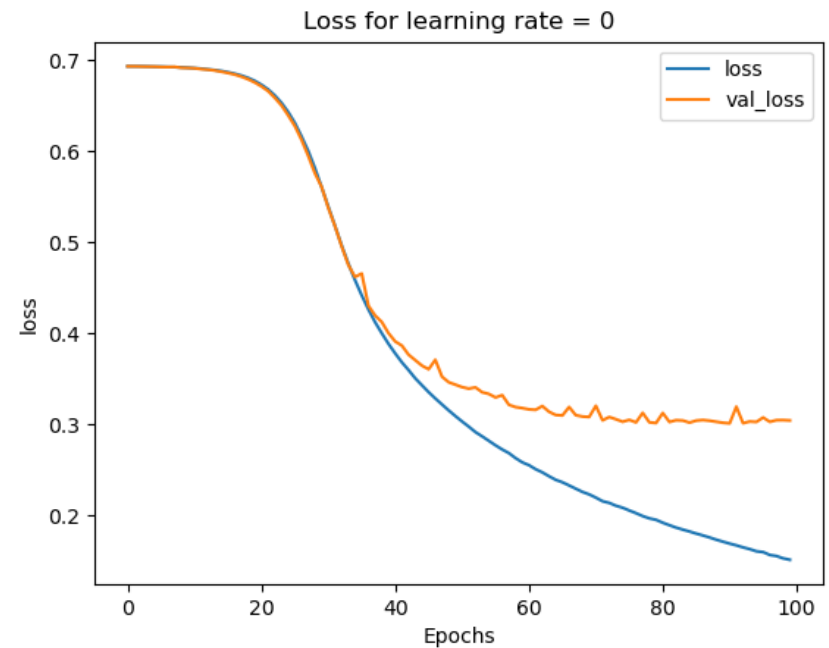
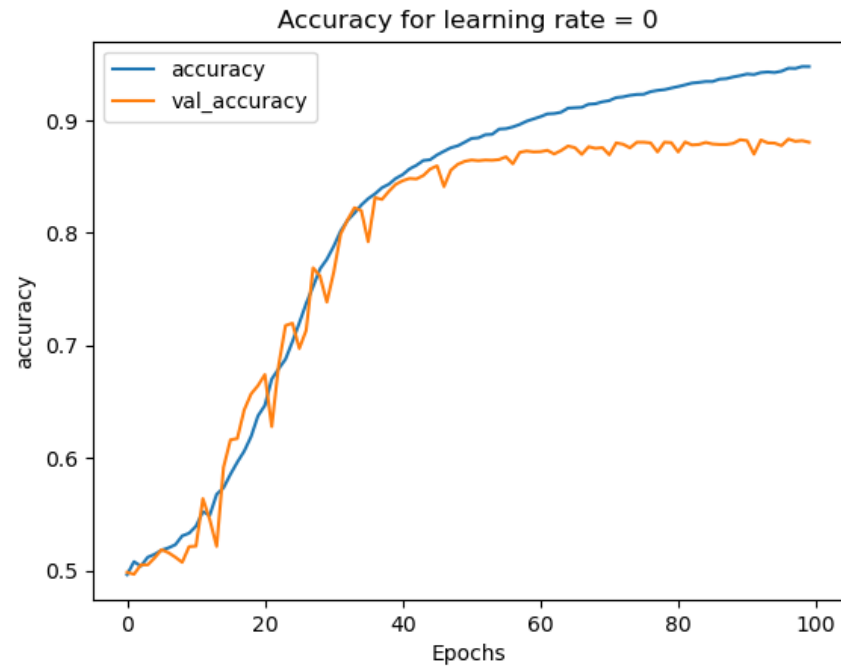
Too high



Too large of a learning rate causes drastic updates which lead to divergent behaviors

Source: jeremyjordan.me

How did adjusting the learning rate turn out?



Project: What else can we try changing?

- Embedding dimension
 - Value of 1024 was chosen arbitrarily; is it a good number?
 - Guide for embedding dimension is to have it as the fourth root of vocab size $\rightarrow 11.9 \sim 12$

Project: What else can we try changing?

- Model architecture
 - A bigger model allows us to learn more complex relationships
 - Will a bigger or smaller model increase the changes of overfitting?

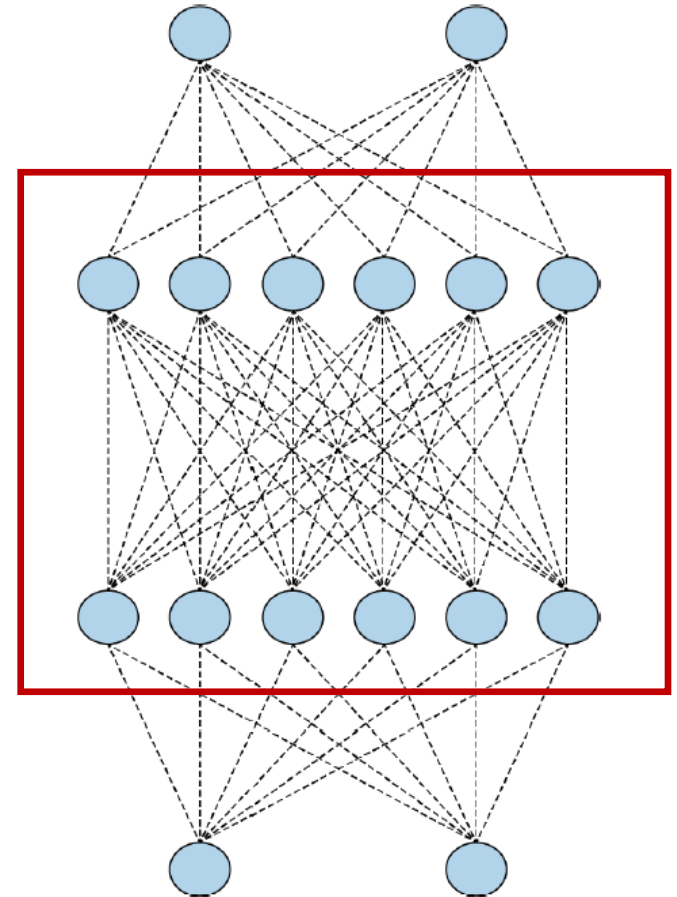
Layer (type)	Output Shape	Param #
=====		
input_1 (InputLayer)	[(None, 600)]	0
embedding (Embedding)	(None, 600, 1024)	20480000
global_average_pooling1d (GlobalAveragePooling1D)	(None, 1024)	0
dense (Dense)	(None, 16)	16400
dense_1 (Dense)	(None, 1)	17
=====		
Total params: 20,496,417		
Trainable params: 20,496,417		
Non-trainable params: 0		

Project: What else can we try changing?

- Using dropout – this is a common technique to reduce overfitting
- As a neural network is trained, each individual neuron is going to have an effect on neurons in the subsequent layers
- Over successive epochs, some neurons can become over-specialised, including neighbouring neurons ending up with similar weights
- This can propagate downstream and lead to the overall network becoming too specialised and overfitting

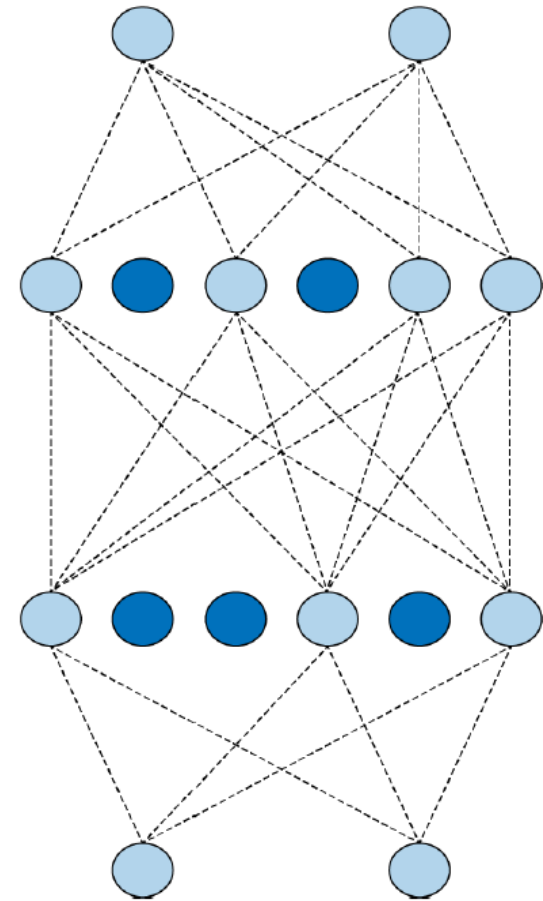
Project: What else can we try changing?

- For example, in the neural network shown here, the neurons in the hidden layers may turn out to have similar weights



Project: What else can we try changing?

- Therefore, while training, we might wish to remove a random number of neurons, effectively ignoring them
- These neurons will no longer therefore contribute to the subsequent layers



Project: What else can we try changing?

- The chances of the network becoming overspecialised should therefore be reduced, and hence it should be better at generalising, and not overfit
- We use a keras layer to implement dropout
- `x = layers.Dense(8, activation = 'relu')(x)`
- `x = layers.Dropout(0.5)(x)`
- The code means that we dropout, at random, 50% of neurons in the specified layer → end up with just 4 neurons in the layer for this example
- It will take some experimentation to find a good percentage for our model

Project: What else can we try changing?

- We can try regularization, which helps limit overfitting by reducing the polarization of weights, i.e. if the weights on some of neurons are too large, regularization will punish them
- Regularization speaks to a common way of addressing overfitting, which is to put constraints on how complex the model is, by forcing its weights to take only small values (overfit models tend to have inflated weights)
- That is to say, make the distribution of weights more regular

Project: What else can we try changing?

- This is known as weight regularization : we add to the loss function of the model, a cost associated with having large weights
- This cost can be in the form of:

L1 regularization

- the cost added is proportional to the absolute value of the weights

L2 regularization

- the cost added is proportional to the square of the value of the weights

Project: What else can we try changing?

- L2 is commonly used for NLP problems, and is specified in the dense layer as :
- `x = layers.Dense(new_layer_dim, activation = 'relu',
kernel_regularizer = keras.regularizers.l2(0.01))(x)`
- The value of 0.01 is the regularization factor, and is another hyperparameter – you can try changing this value to see, typical values between 0 to 0.1 (e.g. 0.1, 0.01, etc)
- Let's go to our notebooks and explore the code chunks for us to do our project

Reviewing model prediction on new review

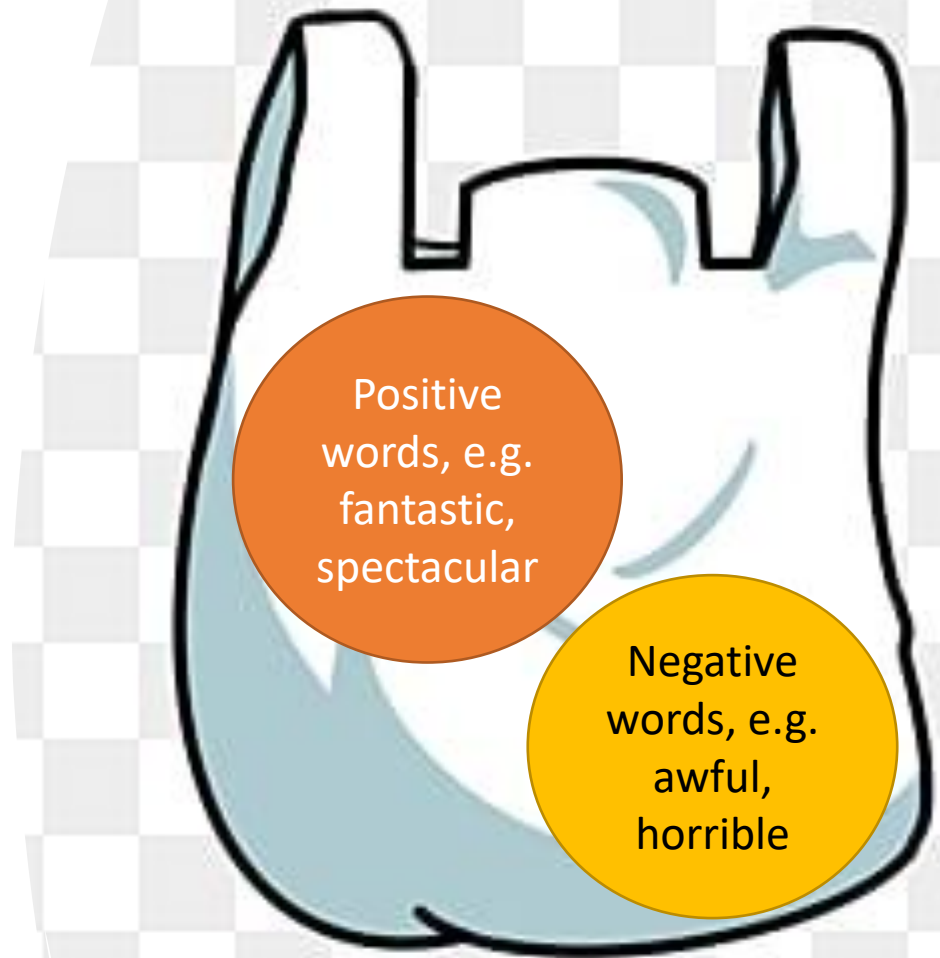
- The result from calling the `predict()` method for our model on the new review was ~ 97
- What does this result mean?

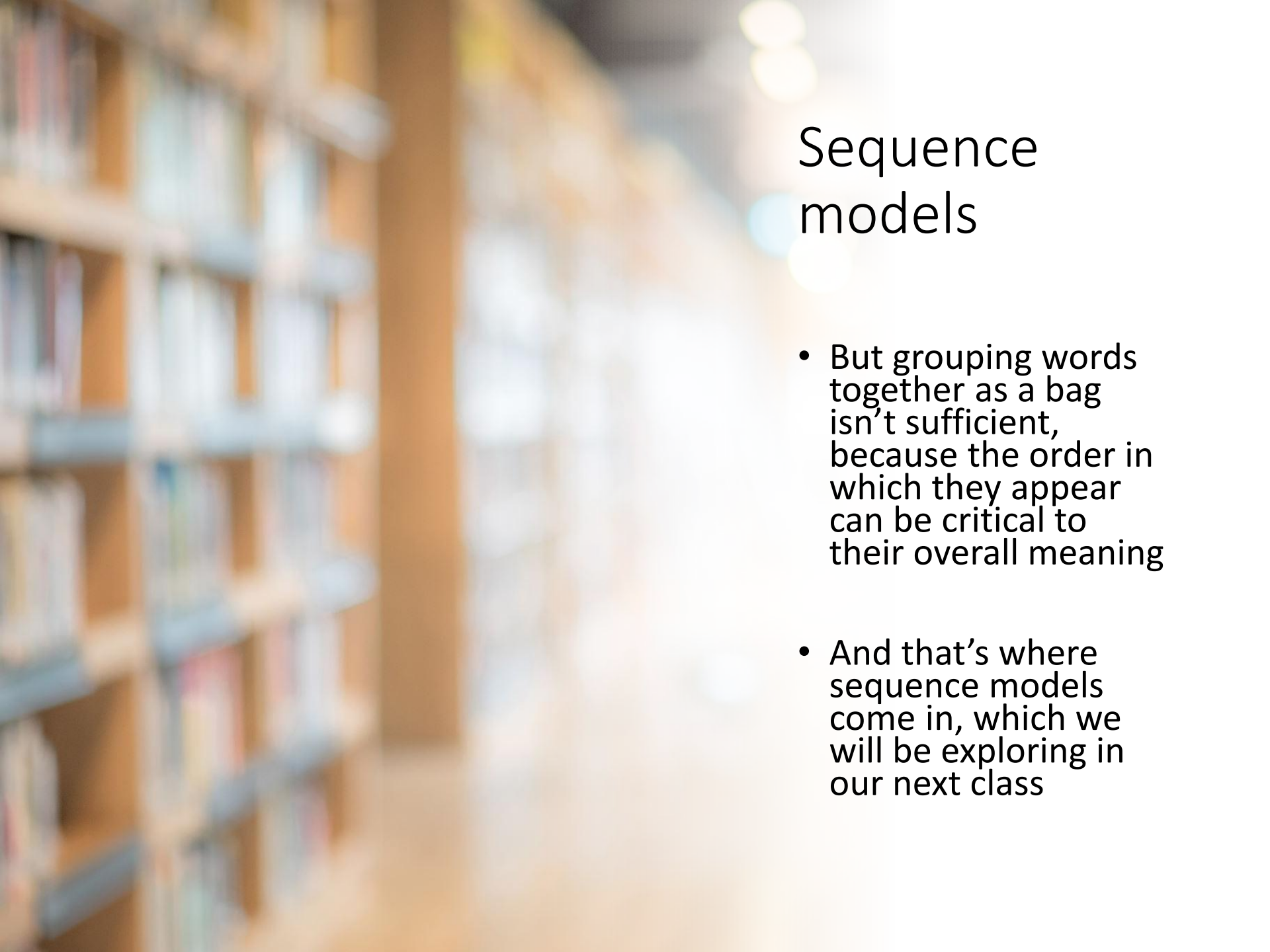
Sequence models

- The approaches we have seen so far, encoding words as integer or even as multi-dimensional vectors, are known as bag-of-words approaches
- We treat the text as a collection of individual tokens, with no consideration of sequence (albeit for embeddings, the tokens are “clustered” together in terms of how related they are, but still without order)

Embeddings

- Nevertheless, the word order is still ignored





Sequence models

- But grouping words together as a bag isn't sufficient, because the order in which they appear can be critical to their overall meaning
- And that's where sequence models come in, which we will be exploring in our next class

End Day 2